Makai Labs — Frontend Architecture Proposal

This design is for analysts who review **10–200 AI-extracted fields** against original documents throughout their working day. I prioritize a fast field-to-source review loop, explicit human decisions, and recoverable work. I used the normal workflow and interruption/concurrency scenarios to choose application boundaries and state owners.

---

1. High-level architecture

I would build one modular application with **Next.js App Router, React, and TypeScript**, containing a document queue and a review workspace.

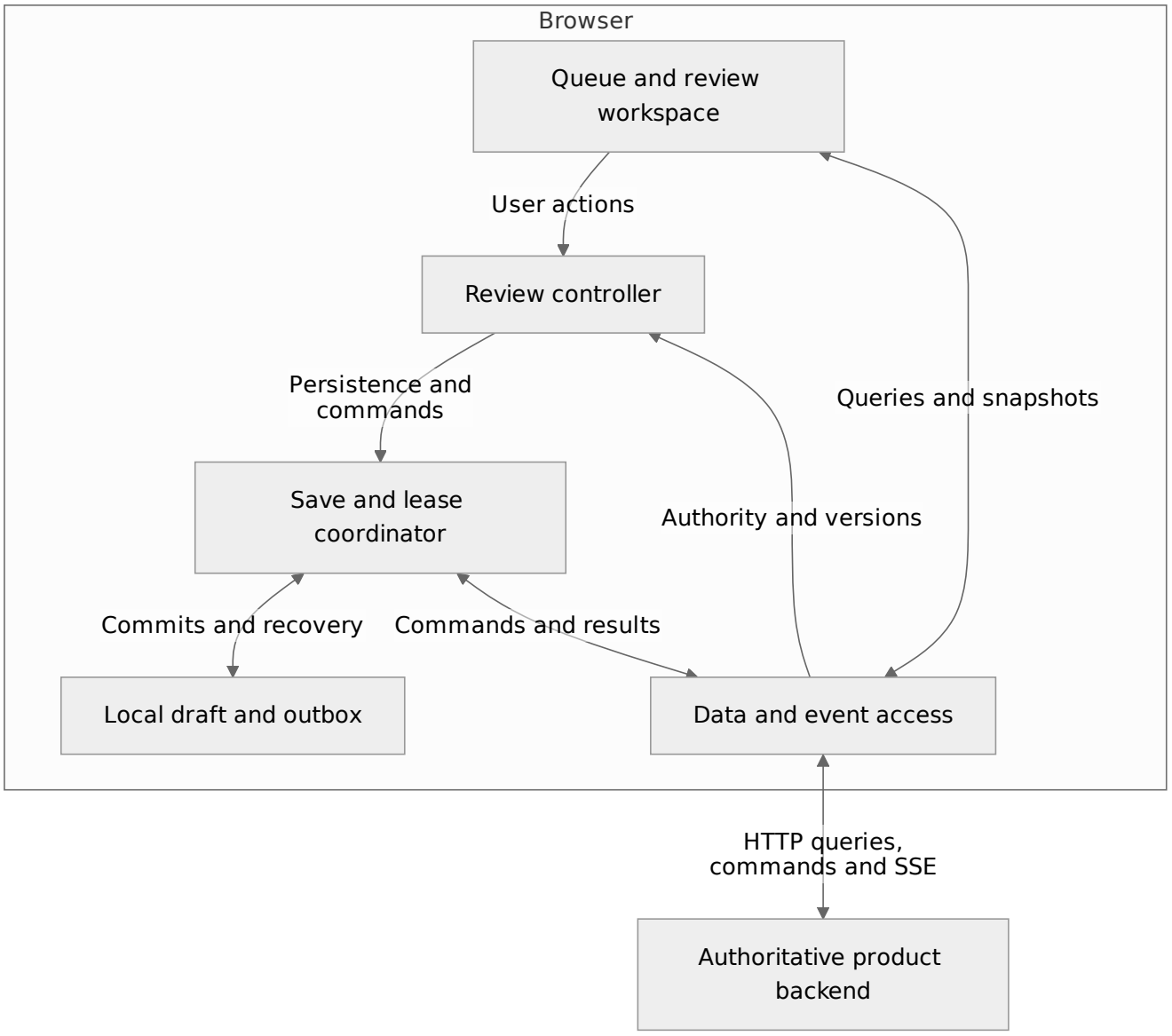
- **Browser:** editing, evidence navigation, local drafts, and synchronization.
- **Next.js:** routing, authenticated entry, shared layouts, and useful initial loading.
- **Product backend:** authoritative permissions, editing reservations, versions, writes, and final decisions.

Next.js aligns with the stack sought for Makai’s role and my React/TypeScript background. Its routing, layouts, and server integration fit the application; the exercise itself does not mandate it. A React/Vite SPA is also viable, with fewer server/client boundaries. Review logic remains independent of routing.

One modular app in a conventional repository is sufficient for this scope. Microfrontends introduce independent composition/deployment boundaries; a monorepo organizes applications and packages within one repository. They address different needs and can coexist. Multiple applications, shared packages, or independent teams would justify revisiting those choices.

In an existing Makai application, these responsibilities would integrate with its routing, identity, UI components, and data conventions. A thin BFF through Route Handlers is optional for server-held credentials, API adaptation, or composition. It adds a hop and streaming responsibility; review authority stays with the product backend.

Responsibility and authority map



The coordinator owns the save/lease protocol; data and event access owns remote IO through shared adapters. The workspace includes the form and original viewer. Next.js authenticated entry and initial loading sit outside this browser runtime.

Security and session boundaries

Three separate checks govern access:

1. **Authentication** establishes identity through the organization’s identity system.
2. **Authorization** determines permitted documents and actions.
3. **An editing lease** temporarily reserves a document for one authorized editing session.

Consultation acquires no lease. Backend authorization covers originals, fields, history, commands, and events, scoped to account, tenant where applicable, resource, and action. Protected routes and disabled controls support the experience; enforcement remains on the backend.

Use HTTPS. Where compatible with identity integration, use server-managed cookies with `HttpOnly`, `Secure`, an appropriate explicit `SameSite` policy, and CSRF protection for cookie-authenticated commands. Tokens and credentials are excluded from local drafts.

Drafts contain minimal values, comparison bases, review metadata, and operation information, separated by account, tenant, document, and editing branch. This is logical isolation, not encryption or protection from browser-profile access. Retention requires an approved policy; full originals and queue copies are excluded.

Render source snippets, extracted values, and recovered drafts as untrusted data. CSP adds protection; routine telemetry excludes document contents and credentials.

Known authentication expiry or permission revocation stops protected work. Recovery requires the correct identity and fresh permissions before displaying retained data or resolving operations. Pending work is never blindly replayed after reauthentication.

---

2. Major components and data/state flow

Component	Responsibility
Document queue	Server-backed search/filter/sort, bounded pages, live ownership/status, and entry into consultation or review.
Review workspace	Fields beside the immutable original, bidirectional highlighting, validation, progress, and keyboard navigation. A viewer adapter contains PDF/image rendering and geometry.
Review controller	Shared semantic actions for buttons and shortcuts, action guards, review invalidation, and pause/finalization preparation.
Save and lease coordinator	Coherent local commits, ordered attempts, selective acknowledgments, uncertain-result recovery, and lease acquisition/renewal/release. Uses the data-access adapters for commands.
Data and event access	HTTP query/command adapters, confirmed snapshots, authorized SSE notifications, polling fallback, and version/sequence-aware reconciliation.

State	Owner
Live inputs and form errors	React Hook Form + Zod
Session phase, selection, and operational review metadata	Zustand, changed through the review controller
Recoverable draft, comparison base, and pending operations	IndexedDB through Dexie
Confirmed server snapshots and queue pages	TanStack Query
Search, filters, and ordering	URL
Unshared visual state	Local React state

Live input, a durable draft, and a confirmed server snapshot are distinct stages with explicit handoffs. Refetches cannot reset dirty inputs, and drafts are not presented as server-confirmed data. Progress and actions derive from document status, authority, review, validation, and persistence state.

The browser runtime lives in the authenticated layout shared by queue and workspace, so navigation does not abandon in-flight operations. Account/tenant changes isolate caches, callbacks, and pending work. Server-side query caches are request-scoped.

I prefer Zustand plus an explicit TypeScript controller for this bounded workflow. XState provides more formal lifecycle modeling but adds a tool I have less practical experience with. The accepted cost is explicitly modeling and verifying asynchronous transitions.

Analyst workflow

1. **Consult or start review.** Reading/prefetching does not reserve a document. Review requires a confirmed lease; another session’s holder is shown with authorized consultation available.
2. **Compare and confirm.** Selection highlights evidence. The analyst explicitly confirms the current value or correction against the source. Confirmation is tied to that value/source revision; later changes invalidate it. Progress covers the full field set, including filtered-out fields.
3. **Save continuously.** Inputs update immediately; the coordinator commits local work and sends identified batches. Confirm-and-next advances after local commit in normal mode, or server acknowledgment in server-only mode.
4. **Pause or prepare a decision.** Freeze the accepted checkpoint, persist it, and resolve pending writes. Return to editing is available before release/final-command dispatch. Normal pause synchronizes and releases the lease. A disclosed local

pause can preserve an unsynchronized durable draft; offline pause stops renewal and waits for server expiry.

5. **Complete explicitly.** Require current authority/version, every field reviewed, no unresolved conflicts or relevant writes, and action-specific rules. Approval requires no blocking validation errors; rejection requires a reason. The backend validates and applies the decision atomically. After confirmation, offer Review next or Return to queue, preserving criteria.

Reviewed and valid are different. An inspected field can retain a known error that blocks approval but permits a policy-valid rejection. Selecting, tabbing, or autosaving does not confirm review. A schema-valid value can differ from the original, and an incorrect human confirmation can still lead to approval. Early rejection of irrevisable documents remains a business policy to validate.

An on-demand history shows server-known actors and review activity, excluding unsynchronized local progress.

---

3. Hardest problems and approach

A. Responsive editing without losing or overwriting work

Problem. Durability, editing authority, and remote outcomes must remain consistent when responses arrive late or disappear. The UI distinguishes editing, saved locally, saved to server, and final decision confirmed.

Approach: a durable draft and ordered synchronization. Dexie commits raw values, review metadata, and pending operations—the **outbox**—coherently. Incomplete input is preserved separately from valid canonical data. Commit continuously during editing, without relying on a tab-close handler. Show Saved locally only after completion; a close before commit can lose the latest input.

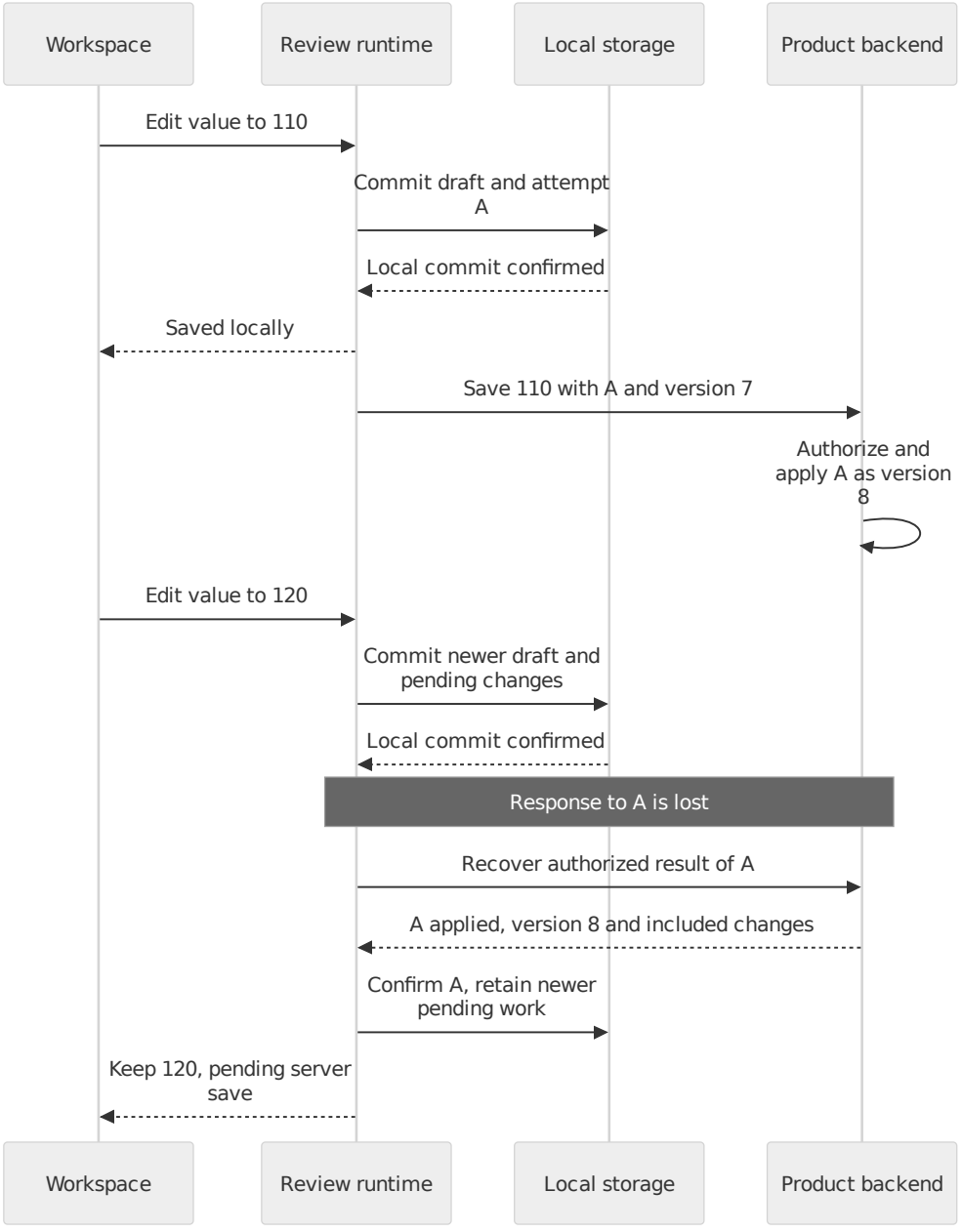
The coordinator follows these rules:

1. In normal mode, persist each attempt before dispatch, including its unique identity, expected document review version, and session/lease context.
2. Send bounded batches with **one unresolved data write per document**. Typing, reads, and renewal may continue under their own guards.
3. Compact only never-sent work. A sent attempt retains its payload and identity.
4. Require atomic backend authorization, lease/version checks, application, and deduplication with a recoverable result.

5. Apply an **acknowledgment** only to its included changes. Recover an uncertain identified result or safely retry the same operation, without creating a replacement. Automatic retries/verification are bounded; retain the draft/identity and offer authorized verification if uncertainty remains.

Source/extraction identities are separate from the review version. Relevant source, schema, or field-set changes cause the backend to advance the review version and publish a coherent snapshot, invalidating affected confirmations. Stale writes or final decisions are rejected. Presence and lease renewal do not change that review version.

For example, an analyst sends **110**, then enters **120**. Recovering a lost response for 110 confirms only its included changes; **120 remains the current pending draft**.



This sequence isolates interrupted-write recovery, the most complex interaction. The runtime includes the coordinator and command adapters. Other flows could be diagrammed; text covers them here to avoid redundant visuals.

Authority during interruption. Temporary exclusivity keeps review responsibility clear. A lease belongs to one editing session; a second tab consults. Normal reassignment requires release or expiry, accepting a wait after abandonment. Brief disconnected editing requires conservatively valid confirmed authority, usable evidence, and permitted persistence. Pause before expiry and revalidate after suspension/reconnection. Instant offline awareness of revocation is not guaranteed.

Recovery and conflicts:

1. Reauthorize and resolve uncertain operations first. A recorded past result does not reapply the write or restore an expired lease.
2. With compatible source/schema/field identities and a reliable base, compare base, local, and remote values and review metadata.
3. Recover non-conflicting work as a draft. Ask the analyst about contradictions and validate the combined result, including rules across fields.
4. Incompatible source changes prevent automatic mapping and invalidate affected confirmations. Terminal documents stay read-only, disclosing unapplied drafts.

Matching values alone do not establish an operation’s result or review authorship. Version/sequence-aware handling covers queries, events, and recovered results: none may roll back a newer snapshot or reset dirty inputs. Lease observations use their appropriate ordering independently of the review version.

Degradation and limits. On local-storage failure, retry boundedly and clean only confirmed, unnecessary data. If permitted, switch to server-only saving through the same coordinator after resolving uncertain attempts. Protection then requires remote acknowledgment; unsent memory-only edits can be lost on closure.

If both destinations fail, pause with memory preservation and retry, disclosing closure risk. An uncertain final decision remains under verification; aborting a request does not cancel a remote effect.

Tradeoff. Local drafts/reconciliation add complexity compared with remote-only saving, but support interruptions and closure. Document-level versions simplify consistency and can cause conservative conflicts. Consider field versions if measured independent conflicts justify them.

B. Reliable evidence review with keyboard and assistive technology

Problem. Analysts must verify the source efficiently, while navigation stays distinct from review and works with keyboard and assistive technology.

Approach: stable evidence references and an accessible workspace. Use **PDF.js behind a viewer adapter** for native/scanned PDFs and an image-page adapter for page images. Evidence requires stable field/evidence identifiers, source revisions, page references, and agreed coordinates. Overlays transform with scale/rotation. Text matching is an auxiliary search aid, not provenance.

- Selecting a field scrolls its source region into view and highlights it without stealing input focus or resetting zoom.
- Selecting a linked source region highlights its field. Go to field explicitly moves editing focus.
- Multiple regions can be traversed; shared regions expose linked fields without guessing.
- Ordinary text selection/copy remains available. Missing links permit disclosed manual comparison when the source is usable.

Use an adjustable desktop split view. Narrow widths and browser zoom use stacked/alternate panels, retaining context and selection. Desktop-first does not assume equivalent phone productivity. Form controls reflow even where the source needs two-dimensional navigation.

Native labels, controls, landmarks, and discoverable contextual shortcuts support the full task. Confirm-and-next follows stable pending-field order and preserves focus through filtering. Shortcuts respect typing, multiline input, composition, and repeated keys; character-only shortcuts need appropriate focus scope or disable/remap support. Text/icons supplement color, and restrained announcements avoid stealing focus.

Dependency and limit. Accessible evidence needs an independent representation of the original. Native text/structure requires reading-order verification; scans need an upstream representation with adequate fidelity. Extracted values cannot prove themselves, and unverifiable source cannot produce a fictitious confirmation.

Verify WCAG 2.1 AA across queue, comparison, editing, recovery, and finalization with keyboard, zoom/reflow, assistive technology, and automated checks. PDF.js alone does not establish compliance.

Tradeoff. Evidence integration, source accessibility, and focus coordination cost effort. Native controls are the starting point; a custom spreadsheet grid adds substantial keyboard and accessibility behavior to maintain.

C. A large, live queue that stays fast and predictable

Problem. Tens of thousands of documents must remain searchable while analysts promptly see committed ownership changes and retain their place.

Approach: bounded server queries and live status. Search/filter/sort run on the server, using URL criteria and bounded pages/caches. Prefer cursor pagination; existing offset pagination is acceptable if sufficient. The browser does not load the entire queue to search it.

Use **SSE for authorized notifications and HTTP for commands**, assuming compatible authentication/hosting. Existing suitable WebSockets can be reused.

- **Ownership/status:** update the row promptly on event receipt, without manual refresh. Known loss of authority stops active editing.
- **Acquisition races:** the backend grants one lease atomically and returns current ownership to the other requester.
- **Order/membership:** preserve focus and scroll; offer results refresh for new/reordered items. Rows awaiting removal still show actual unavailable/terminal status.

On stream failure, poll the visible queue and active document, label updates as delayed, and reconcile on reconnect. Missed events require replay or current snapshots. If requests also fail, disclose cached data as stale rather than showing an empty queue. Transport loss alone neither grants nor revokes a lease.

Tradeoff and limit. Push depends on integration support; polling has lower freshness. Live status helps coordination while atomic acquisition prevents simultaneous editing.

Performance and scalability

Area	Baseline and measured response
Editing	Scoped form/store subscriptions. Measure input response and React commit cost before broader optimization.
Queue	Bound payloads, pages, and caches. Separate query/network latency from rendering. Optimize backend queries as needed; consider TanStack Virtual for a measured DOM bottleneck, preserving semantics/focus.
Source viewer	Lazily render needed/nearby pages using PDF.js workers; release obsolete tasks/resources. Measure evidence readiness, scrolling, and memory.
Long sessions/events	Bound retained data; coalesce observational patch/refetch work without delaying ownership status. Measure memory growth, request/event volume, and recovery.

Pagination bounds fetching; virtualization bounds mounted rows. They can coexist, but virtualization does not fix slow queries or unlimited retained data. I would not virtualize 200 fields solely because that count is possible.

Profile representative documents, large/scanned PDFs, queue sizes, event bursts, slower connections, and long sessions, pairing performance with keyboard/assistive checks.

---

4. Assumptions, limits, and next steps

These are contracts to validate with the team, not claims about existing Makai APIs. I assume originals arrive as **PDFs or page images**; other formats need an upstream, faithful reviewable representation linked to the original and its evidence.

Assumption/dependency	Consequence if unavailable
Authorized operations, atomic leases, version-checked writes, recoverable idempotent results, and authoritative finalization. Result retention supports the recovery window.	Backend work is required. Preserve and pause where outcomes cannot be established; frontend flags cannot replace enforcement.
Coherent source/schema/extraction revisions, stable evidence identities, and a complete declared field set.	Missing links permit manual inspection. Incompatible revisions block automatic reconciliation. Coverage cannot discover omitted fields.
Remote drafts support intermediate values and review metadata separately from canonical data.	Unsupported partial values remain locally protected only where committed/permitted. Remote handoff and server-only recovery have reduced coverage.
Policy permits minimal local drafts on suitable profiles/devices with agreed retention.	Server-only mode cannot recover unsent edits after closure. The gap against interruption recovery needs product/security agreement; pending drafts are not silently discarded.
Accessible originals and validation/rejection/escalation rules.	Source remediation and business policy are needed. Full field review is the baseline; early rejection requires explicit policy.
Supported browsers, representative usage, and compatible event authentication/hosting.	Validate viewer/storage/accessibility/performance and integration. Polling without push has disclosed lower freshness.

Recovery guarantee and scope

After a confirmed local commit, pending work is recoverable in the **same browser profile**, while stored data remains available and access is revalidated. Another device recovers only server-confirmed work. Uncommitted input, deleted profile data, and

device failure are outside that guarantee. Recovered corrections do not guarantee an offline original or editing authority.

Excluded: simultaneous coediting, indefinite offline editing, original modification, frontend OCR/model execution, and detailed backend infrastructure. No application implementation, performance benchmarks, or accessibility conformance tests have been performed.

Proposed next steps

1. **Validate contracts:** backend lease/recovery semantics, source references/accessibility, identity, local-data policy, and business rules.
2. **Prototype critical interactions:** source/field navigation, confirmation, keyboard focus, and responsive comparison.
3. **Build one vertical slice:** queue acquisition through final decision, verifying concurrency, lost responses, interruption, and recovery.
4. **Measure representative use:** accessibility, input response, viewer behavior, queue latency, and long-session memory; refine where evidence warrants it.

Optional supporting material: GitHub repository. This proposal answers the challenge independently.

AI assistance. I used ChatGPT to compare alternatives, challenge failure scenarios, and organize/draft this proposal. I reviewed and selected the decisions and remain responsible for the proposal.